

JavaScript Strings Review

String Basics

- **Definition:** A string is a sequence of characters wrapped in either single quotes, double quotes or backticks. Strings are primitive data types and they are immutable. Immutability means that once a string is created, it cannot be changed.
- **Accessing Characters from a String:** To access a character from a string you can use bracket notation and pass in the index number. An index is the position of a character within a string, and it is zero-based.

```
1 const developer = "Jessica";  
2 console.log(developer[0]); // J
```

"J"



- `\n` (Newline Character): You can create a newline in a string by using the `\n` newline character.

```
1 const poem = "Roses are red,\nViolets are blue,\nJavaScript is fun,\nAnd so are you";
2 console.log(poem);
```

"Roses are red,
Violets are blue,
JavaScript is fun,
And so are you"

- **Escaping Strings:** You can escape characters in a string by placing backslashes (`\`) in front of the quotes.



Template Literals (Template Strings) and String Interpolation

- **Definition:** Template literals are defined with backticks (`). They allow for easier string manipulation, including embedding variables directly inside a string, a feature known as string interpolation.

```
1  const name = "Jessica";  
2  const greeting = `Hello, ${name}!`;   
3  console.log(greeting); // "Hello, Jessica!"
```



ASCII, the `charCodeAt( )` Method and the `fromCharCode( )` Method

- **ASCII:** ASCII (American Standard Code for Information Interchange) is a character encoding standard used to represent basic English characters using numeric values. Earlier lessons introduce `charCodeAt( )` and `fromCharCode( )` using ASCII examples.
- **Unicode:** JavaScript strings use Unicode internally, specifically UTF-16 encoding. For the first 128 characters (basic Latin letters, digits, and common symbols), the Unicode values match ASCII codes. This is why ASCII-based examples continue to work in JavaScript.
- **The `charCodeAt( )` Method:** This method returns the UTF-16 code unit of the character at a specified index. For basic Latin characters, this value matches the ASCII code.

```
1 const letter = "A";  
2 console.log(letter.charCodeAt(0)); // 65
```



- The `fromCharCode()` Method: This method converts an ASCII code into its corresponding character.

```
1 const char = String.fromCharCode(65);  
2 console.log(char); // A
```



string. If the substring is found, `indexOf` returns the index (or position) of the first occurrence of that substring. If the substring is not found, `indexOf` returns `-1`, which indicates that the search was unsuccessful.

```
1 const text = "The quick brown fox jumps over the lazy dog"
2 console.log(text.indexOf("fox")); // 16
3 console.log(text.indexOf("cat")); // -1
```

- **The `includes()` Method:** This method is used to check if a string contains a specific substring. If the substring is found within the string, the method returns `true`. Otherwise, it returns `false`.

```
1 const text = "The quick brown fox jumps over the lazy dog"
```



- **The `slice()` Method:** This method extracts a portion of a string and returns a new string, without modifying the original string. It takes two parameters: the starting index and the optional ending index.

```
1  const text = "freeCodeCamp";
2  console.log(text.slice(0, 4)); // "free"
3  console.log(text.slice(4, 8)); // "Code"
4  console.log(text.slice(8, 12)); // "Camp"
```



- The `toUpperCase()` Method: This method converts all the characters to uppercase letters and returns a new string with all uppercase characters.

```
1 const text = "Hello, world!";  
2 console.log(text.toUpperCase()); // "HELLO, WORLD!"
```

- The `toLowerCase()` Method: This method converts all characters in a string to lowercase.



- The **replace()** Method: This method allows you to find a specified value (like a word or character) in a string and replace it with another value. The method returns a new string with the replacement and leaves the original unchanged because JavaScript strings are immutable.

```
1 const text = "I like cats";  
2 console.log(text.replace("cats", "dogs")); // "I like dog"
```



- The `replaceAll()` Method: This method allows you to find all occurrences of a specified value (a word, character, or pattern) in a string and replace them with another value. It works like `replace()`, but instead of stopping after the first match, it updates every match found in the string.

```
1 const text = "I love cats and cats are so much fun!";  
2 console.log(text.replaceAll("cats", "dogs")); // "I love dogs and dogs are so much fun!"
```



```
2 console.log(text.repeat(3)); // "HelloHelloHello"
```

- The `trim()` Method: This method is used to remove whitespaces from both the beginning and the end of a string.

```
1 const text = "  Hello, world!  ";  
2 console.log(text.trim()); // "Hello, world!"
```



- The `trimStart()` Method: This method removes whitespaces from the beginning (or "start") of the string.

```
1 const text = " Hello, world! ";  
2 console.log(text.trimStart()); // "Hello, world! "
```



```
2 console.log(text.trimEnd()); // " Hello, world!"
```

- The **prompt()** Method: This method of the **window** is used to get information from a user through the form of a dialog box. This method takes two arguments. The first argument is the message which will appear inside the dialog box, typically prompting the user to enter information. The second one is a default value which is optional and will fill the input field initially.



☐ Review the JavaScript Strings topics and concepts.

Please complete the assignment

Submit

Ask for Help